

TuniRoute

Cahier des Charges Fonctionnel Application de navigation des transports en commun en Tunisie

Encadrant	M. Mansouri Oussema
Équipe	Projet de fin d'études – Génie Logiciel
Année	2025 – 2026
Version	1.0
Date	Mai 2026

Ce document présente le cahier des charges fonctionnel de l'application **TuniRoute**, une solution de navigation dédiée aux transports en commun en Tunisie (métro, bus, TGM, louage). Il décrit le contexte du projet, les acteurs impliqués, les exigences fonctionnelles et non-fonctionnelles, ainsi que les diagrammes UML illustrant l'architecture logicielle.

1. Présentation du Projet

1.1 Contexte général

La Tunisie dispose d'un réseau de transports en commun varié — métro léger, bus urbains, TGM (Train de banlieue Tunis-Goulette-Marsa) et louages interurbains — mais souffre d'un manque d'outils numériques accessibles permettant aux usagers de planifier leurs déplacements efficacement. La majorité des applications existantes ne couvrent pas l'ensemble des modes de transport ou ne proposent pas d'itinéraires multimodaux.

1.2 Objectifs du projet

- Proposer une application mobile et web de navigation multimodale pour les transports en commun tunisiens.
- Permettre à l'utilisateur de rechercher un itinéraire entre deux points avec estimation du temps et des alternatives.
- Offrir une interface d'administration pour la gestion des lignes, stations et horaires.
- Garantir une expérience utilisateur intuitive, performante et accessible.

1.3 Périmètre

Le projet couvre les villes et agglomérations desservies par les réseaux Transtu (métro et bus), SNCFT (TGM) et les lignes de louage nationales. La première version cible Tunis et sa grande banlieue.

2. Acteurs du Système

Utilisateur

Tout usager souhaitant planifier un déplacement en transport en commun. Il peut rechercher des itinéraires, consulter son historique et gérer ses favoris.

Administrateur

Responsable de la maintenance du système : gestion des lignes, stations et horaires via un tableau de bord dédié.

3. Exigences Fonctionnelles

3.1 Authentification

- Inscription avec email, nom, prénom et mot de passe.
- Connexion sécurisée (JWT) pour l'accès à toutes les fonctionnalités.
- Gestion des rôles : Utilisateur / Administrateur.

3.2 Recherche d'itinéraire

- Saisie du point de départ et de la destination.
- Calcul automatique des itinéraires optimaux (temps, correspondances).

- Estimation de la durée totale du trajet.
- Affichage des résultats avec détail des correspondances.
- Proposition d'itinéraires alternatifs.

3.3 Historique et Favoris

- Consultation de l'historique des recherches.
- Ajout, consultation et suppression des itinéraires favoris.

3.4 Administration

- Gestion des lignes de transport (ajout, modification, suppression).
- Gestion des stations (ajout, modification, suppression).
- Gestion des horaires (ajout, modification, suppression).

4. Exigences Non-Fonctionnelles

Performance	Temps de réponse inférieur à 2 secondes pour le calcul d'itinéraire.
Disponibilité	Taux de disponibilité cible : 99,5 % (hors maintenance planifiée).
Sécurité	Authentification JWT, hachage bcrypt des mots de passe, HTTPS.
Scalabilité	Architecture microservices permettant une montée en charge horizontale.
Accessibilité	Interface responsive, compatible mobile et desktop.
Maintenabilité	Code documenté, tests unitaires avec couverture ≥ 80 %.

5. Architecture Technique

Frontend	React.js / React Native, TailwindCSS
Backend	Spring Boot (Java), REST API
Base de données	PostgreSQL + PostGIS
Authentification	Spring Security + JWT
Déploiement	Docker, Kubernetes, GitHub Actions (CI/CD)
Cartographie	OpenStreetMap, Leaflet.js

6. Diagrammes UML

6.1 Diagramme de cas d'utilisation

Ce diagramme présente les interactions entre les acteurs et le système. L'**Utilisateur** et l'**Administrateur** s'authentifient avant d'accéder à leurs fonctionnalités respectives, toutes liées par des relations «*include*».

Diagramme de cas d'utilisation - TuniRoute

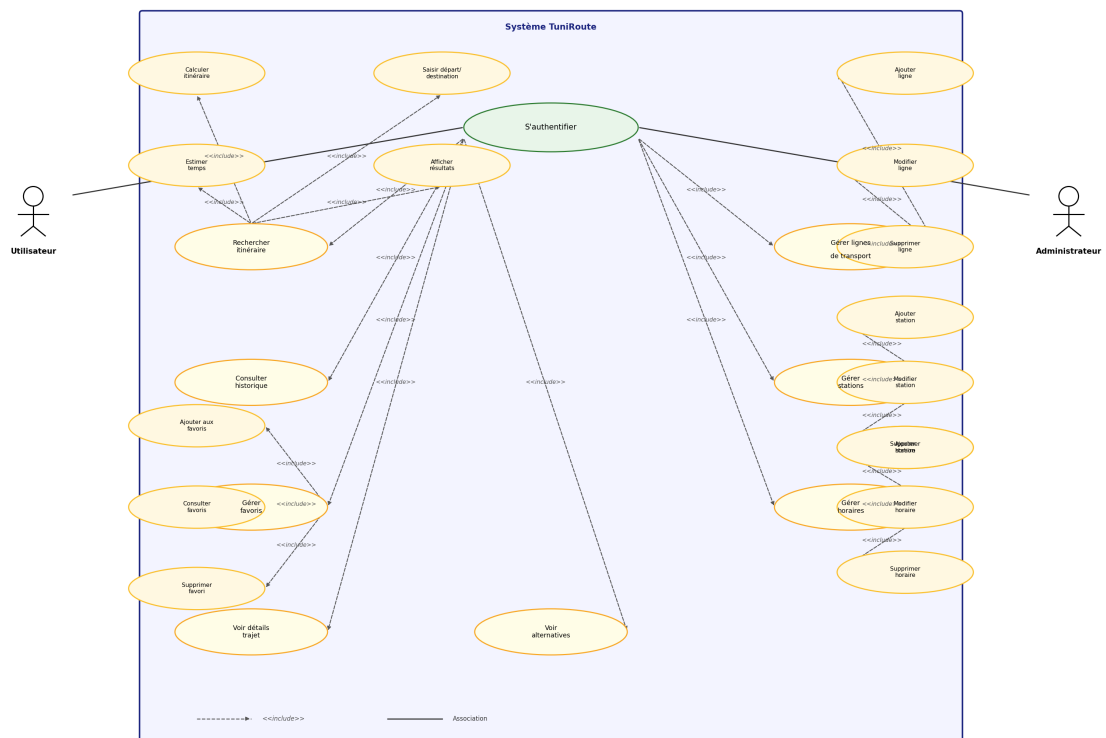


Figure 1 – Diagramme de cas d'utilisation TuniRoute

6.2 Diagramme de classe

Le diagramme de classe décrit la structure statique du système. Les classes principales sont : **Utilisateur**, **Trajet**, **Itinéraire**, **LigneTransport**, **Station**, **Horaire**, **TypeTransport**, **Role**, **Favori** et **Historique**. La classe *Trajet* est directement reliée à *Station* via les rôles *départ* et *arrivée*.

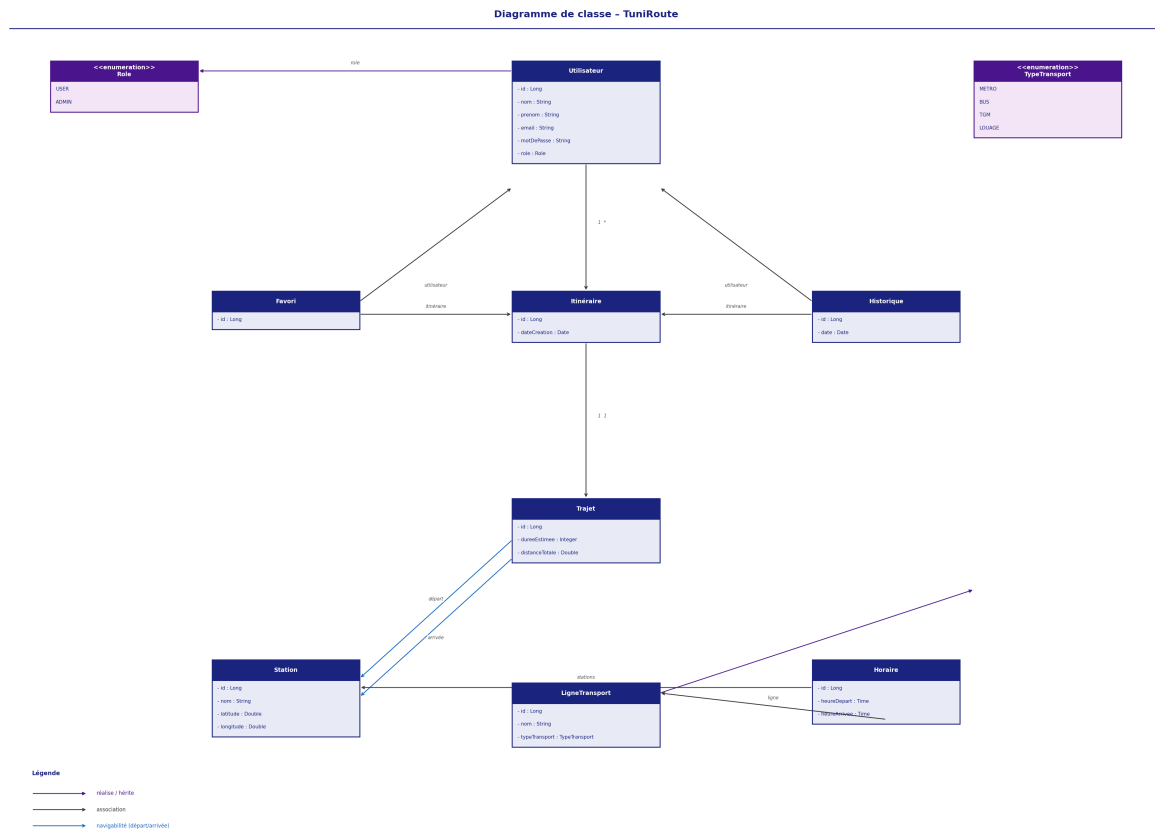


Figure 2 – Diagramme de classe TuniRoute

7. Planning Prévisionnel

Phase	Durée	Livrables
Phase 1 – Analyse	Semaines 1-2	Recueil des besoins, rédaction du cahier des charges
Phase 2 – Conception	Semaines 3-4	Diagrammes UML, architecture technique
Phase 3 – Développement	Semaines 5-10	Implémentation backend et frontend
Phase 4 – Tests & QA	Semaines 11-12	Tests unitaires, tests d'intégration
Phase 5 – Déploiement	Semaine 13	Mise en production (Docker / Cloud)
Phase 6 – Documentation	Semaine 14	Rapport final, présentation